

AGV Controller SDK API 接口文档

1. 概述

`AGVController` SDK 提供了用于控制和管理 AGV 的开发接口。本 SDK 采用 C++14 标准开发，同时提供了跨平台（Windows / Linux）的动态库导出机制，并封装了标准的 C-API 接口，便于 Python（`ctypes`）、C 语言及其他语言进行绑定与集成。

核心特性

- **双层接口架构**：面向对象的 C++ 接口与平铺式 C API 导出接口。
- **线程安全设计**：内部数据（如 Pose 姿态更新）通过 `std::mutex` 保护；通信与超时采用 `std::condition_variable` 进行异步等待同步。
- **后台自动化**：内置独立的数据接收线程（`receiveLoop`）与心跳维持线程（`heartbeatLoop`）。

2. 基础数据结构与枚举

2.1 AGVPose / C_AGVPose (姿态数据)

用于描述 AGV 当前在空间中的位置与姿态。

C++ 结构体

```
namespace agv_sdk {
    struct AGVPose {
        double x{0.0};      // X 轴空间坐标 (m)
        double y{0.0};      // Y 轴空间坐标 (m)
        double z{0.0};      // Z 轴空间坐标 (m)
        double roll{0.0};   // 前方距离 (m) / 翻滚角
        double pitch{0.0};  // 左侧距离 (m) / 俯仰角
        double yaw{0.0};    // 偏航角 (rad)
    };
}
```

C 结构体

```
typedef struct {  
    double x;  
    double y;  
    double z;  
    double roll;  
    double pitch;  
    double yaw;  
} C_AGVPose;
```

2.2 ControlMode / C_ControlMode (控制模式)

定义 AGV 的工作模式。

C++ 枚举

```
namespace agv_sdk {  
    enum class ControlMode {  
        Manual = 2, // 手动控制模式  
        Auto   = 3  // 自动控制模式  
    };  
}
```

C 枚举

```
typedef enum {  
    C_CONTROL_MODE_MANUAL = 2, // 手动控制模式  
    C_CONTROL_MODE_AUTO   = 3  // 自动控制模式  
} C_ControlMode;
```

3. C++ API 接口说明

C++ 接口位于命名空间 `agv_sdk` 中，主要类为 `AGVController`。注：该类禁用了拷贝构造函数与拷贝赋值运算符 (Non-copyable)。

3.1 构造与析构

```
AGVController();  
~AGVController();
```

- **描述**：创建或销毁 `AGVController` 实例。析构时会自动断开连接并停止内部工作线程。

3.2 连接与会话管理

connectAGV

```
bool connectAGV(const std::string& ip, int port);
```

- **功能**：通过 TCP 套接字建立与 AGV 控制器的网络连接，并启动后台接收与心跳线程。
- **参数**：
 - `ip`：AGV 控制器的 IP 地址（如 `"192.168.1.100"`）。
 - `port`：网络端口号。
- **返回值**：连接成功返回 `true`，失败返回 `false`。

disconnectAGV

```
void disconnectAGV();
```

- **功能**：主动断开与 AGV 的连接，停止内部工作线程，释放网络套接字资源。

login

```
bool login(const std::string& username, const std::string& password_hash);
```

- **功能**：向 AGV 发送身份验证请求，建立合法操作会话。
- **参数**：
 - `username`：用户名。
 - `password_hash`：加密后的密码哈希值。
- **返回值**：登录验证成功返回 `true`，失败返回 `false`。

logout

```
bool logout();
```

- **功能**：注销当前登录会话。

- 返回值：注销成功返回 `true`，失败返回 `false`。

3.3 运动与控制指令

`switchControlMode`

```
bool switchControlMode(ControlMode mode);
```

- 功能：切换 AGV 的运行模式（手动 / 自动）。
- 参数：
 - `mode`：控制模式枚举 `ControlMode::Manual` 或 `ControlMode::Auto`。
- 返回值：模式切换请求成功返回 `true`，失败返回 `false`。

`goForward`

```
bool goForward(double dist_mm, int timeout_ms = 30000);
```

- 功能：控制 AGV 直行前进到底距离前方面距离。该接口为阻塞式调用，直到动作完成或超时。
- 参数：
 - `dist_mm`：前进距离，单位：毫米（mm）。
 - `timeout_ms`：超时等待时间，单位：毫秒（ms），默认值 `30000` ms。
- 返回值：在超时时间内成功到达指定距离返回 `true`，执行超时或失败返回 `false`。

`goBack`

```
bool goBack(double dist_mm, int timeout_ms = 30000);
```

- 功能：控制 AGV 直行前进(+)/后退(-)指定距离。该接口为阻塞式调用。
- 参数：
 - `dist_mm`：后退距离，单位：毫米（mm）。
 - `timeout_ms`：超时等待时间，单位：毫秒（ms），默认值 `30000` ms。
- 返回值：成功完成返回 `true`，超时或失败返回 `false`。

`manualCtlVelSet`

```
bool manualCtlVelSet(float vx, float vy, float w);
```

- 功能：在手动控制模式下，实时设定 AGV 的三轴运动速度。
- 参数：

- `vx`: 前向线速度 (mm/s)。
 - `vy`: 侧向线速度 (mm/s)。
 - `w`: 旋转角速度 (rad/s)。
 - 返回值: 速度指令发送成功返回 `true`, 失败返回 `false`。
-

3.4 状态查询与维护

`querySystemState`

```
bool querySystemState();
```

- 功能: 向 AGV 主控发送系统状态查询请求。
- 返回值: 查询指令发送成功返回 `true`, 失败返回 `false`。

`sendHeartBeatsMsg`

```
bool sendHeartBeatsMsg();
```

- 功能: 手动触发发送一次心跳数据包 (注: 控制器内部已包含自动心跳线程, 通常无需手动频繁调用)。
- 返回值: 发送成功返回 `true`, 失败返回 `false`。

`getPose`

```
AGVPose getPose() const;
```

- 功能: 线程安全地获取当前最新的 AGV 姿态数据。
 - 返回值: 返回 `AGVPose` 结构体副本。
-

4. C-API 接口说明 (用于 Python / C 语言调用)

导出函数统一采用 C 声明, 避免名称修饰 (Name Mangling), 支持 Windows `__declspec(dllexport)` 及 Linux `__attribute__((visibility("default")))`。

4.1 句柄与生命周期管理

函数原型	说明	参数 / 返回值
<code>AGV_Handle AGV_Create();</code>	创建 AGV 控制器实例	返回: 成功返回控制器句柄 <code>AGV_Handle</code> (即 <code>void *</code>), 失败返回 <code>NULL</code>
<code>void AGV_Destroy(AGV_Handle handle);</code>	销毁控制器实例	handle: <code>AGV_Create</code> 返回的句柄

4.2 连接与会话接口

函数原型	说明	参数 / 返回值
<code>bool AGV_Connect(AGV_Handle handle, const char* ip, int port);</code>	建立连接	<code>ip</code> : 字符串格式 IP <code>port</code> : 端口号 返回: <code>true</code> 成功, <code>false</code> 失败
<code>void AGV_Disconnect(AGV_Handle handle);</code>	断开连接	<code>handle</code> : 控制器句柄
<code>bool AGV_Login(AGV_Handle handle, const char* username, const char* password_hash);</code>	登录鉴权	<code>username</code> : 用户名 <code>password_hash</code> : 密码哈希 返回: <code>true</code> 成功
<code>bool AGV_Logout(AGV_Handle handle);</code>	注销登录	<code>handle</code> : 控制器句柄 返回: <code>true</code> 成功

4.3 控制指令接口

函数原型	说明	参数 / 返回值
<code>bool AGV_SwitchControlMode(AGV_Handle handle, C_ControlMode mode);</code>	切换模式	<code>mode</code> : <code>C_CONTROL_MODE_MANUAL</code> 或 <code>C_CONTROL_MODE_AUTO</code>
<code>bool AGV_GoForward(AGV_Handle handle, double dist_mm, int timeout_ms);</code>	直行前进 (阻塞)	<code>dist_mm</code> : 距离(mm) <code>timeout_ms</code> : 超时(ms)
<code>bool AGV_GoBack(AGV_Handle handle, double dist_mm, int timeout_ms);</code>	直行后退 (阻塞)	<code>dist_mm</code> : 距离(mm) <code>timeout_ms</code> : 超时(ms)
<code>bool AGV_ManualCtlVelSet(AGV_Handle handle, float vx, float vy, float w);</code>	设置手动速度	<code>vx</code> : 线速度 X <code>vy</code> : 线速度 Y <code>w</code> : 角速度

4.4 状态与姿态接口

函数原型	说明	参数 / 返回值
<code>bool AGV_QuerySystemState (AGV_Handle handle);</code>	查询系统状态	返回: 发送结果 <code>true</code> / <code>false</code>
<code>bool AGV_SendHeartBeatsMsg (AGV_Handle handle);</code>	发送心跳包	返回: 发送结果 <code>true</code> / <code>false</code>
<code>C_AGVPose AGV_GetPose (AGV_Handle handle);</code>	获取姿态数据	返回: <code>C_AGVPose</code> 结构体

5. Python 绑定调用示例 (ctypes)

```
import ctypes

from ctypes import Structure, c_double, c_int, c_float, c_char_p, c_bool, c_void_p


# 1. 定义 C 结构体与枚举
class C_AGV_Pose(Structure):
    _fields_ = [
        ("x", c_double),
        ("y", c_double),
        ("z", c_double),
        ("roll", c_double),
        ("pitch", c_double),
        ("yaw", c_double),
    ]

C_CONTROL_MODE_MANUAL = 2
C_CONTROL_MODE_AUTO = 3


# 2. 加载动态库
lib = ctypes.CDLL("./libagv_controller.so") # Windows 下为 agv_controller.dll


# 3. 函数签名声明
lib.AGV_Create.restype = c_void_p
lib.AGV_Destroy.argtypes = [c_void_p]

lib.AGV_Connect.argtypes = [c_void_p, c_char_p, c_int]
lib.AGV_Connect.restype = c_bool

lib.AGV_Login.argtypes = [c_void_p, c_char_p, c_char_p]
lib.AGV_Login.restype = c_bool

lib.AGV_GoForward.argtypes = [c_void_p, c_double, c_int]
lib.AGV_GoForward.restype = c_bool

lib.AGV_GetPose.argtypes = [c_void_p]
lib.AGV_GetPose.restype = C_AGV_Pose


# 4. 调用逻辑
handle = lib.AGV_Create()

if lib.AGV_Connect(handle, b"192.168.1.100", 8080):
    if lib.AGV_Login(handle, b"admin", b"e10adc3949ba59abbe56e057f20f883e"):
```



```
# 前进 1000mm, 超时时间 10000ms
lib.AGV_GoForward(handle, 1000.0, 10000)

# 获取姿态
pose = lib.AGV_GetPose(handle)
print(f"Current Pose: X={pose.x}, Y={pose.y}, Yaw={pose.yaw}")

lib.AGV_Disconnect(handle)

lib.AGV_Destroy(handle)
```